

UNIT-II PART-1

Data Structures and Algorithm

Data Structures and Algorithms(DSA):

Data Structures and Algorithms are fundamental concepts in computer science that help in organizing, managing, and storing data efficiently for optimal problem-solving.

Data Structures :

A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. Different data structures are suited to different types of applications. The main idea behind using data structures is to minimize the time and space complexities.

An efficient data structure takes minimum space and requires minimum time to execute the data.

It is used for processing, retrieving and traversing.

Classification Of Data Structures :

1. Linear data structure-
2. Non-linear data structure:trees and graphs

Linear data structure:

Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Example: list,tuples,set,dictionaries,stack queue,arrays...etc

Non-Linear Data Structure:

Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run only.

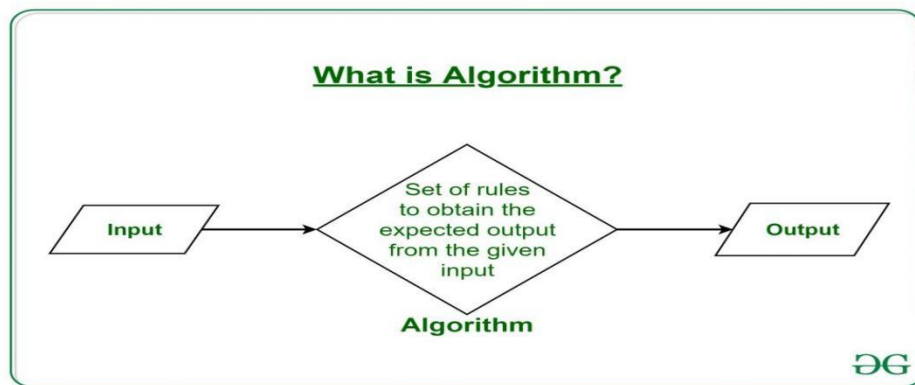
Examples: Trees and Graphs.

Acts As Both Data Structures And Data types:

List,tuples,sets,dictionaries,strings acts as both data structures and data types. As we can add, remove, traverse and retrieve the elements from these things.

ALGORITHMS:

An algorithm is a step-by step procedure or set of rules to solve a specific problem. It takes input,processes it and produces the output.



Time and Space Complexity:

- To solve problems we write programs in computer science.
- If **P** is a program ,then there might be any number of algorithms or solutions possible.

Algorithms are analyzed based on:

Time complexity:

It tells us how fast or slow an algorithm performs relative to input size or how much time is required to solve a problem

Space complexity: how much space is required to solve a problem.or

it is the amount of extra space required.the input is not counted as extra space ,it is mandatory and can't be altered. If no extra space is required then it is called **INPLACE** algorithm.

Basic Algorithms:

Searching:

Searching is the process of finding the location (index) of a specific element (or value) in a collection of data, like a list or array. To determine whether a given element exists in the data and if so, where.

Examples of Searching Algorithms:

Linear Search ($O(n)$) – Check each element one by one.

Binary Search ($O(\log n)$) – Divide and conquer (only works on sorted data).

Sorting:

Sorting is the process of arranging the elements of a collection (like a list or array) in a particular order, usually ascending or descending. To reorder data to make it easier to search, analyze, or display.

Examples of Sorting Algorithms:

Bubble Sort ($O(n^2)$) – Swap adjacent elements if they are in the wrong order.

Selection Sort ($O(n^2)$) – Select the minimum element and place it at the beginning.

Asymptotic notations:

- **Asymptotic analysis** is the study of how an algorithm's performance changes as the input size grows.
- The efficiency of an algorithm depends on the time, memory, and resources it uses.
- We use **asymptotic notations** to measure this efficiency for large inputs.
- These notations describe how fast an algorithm runs when input becomes very large.
- The three main asymptotic notations are:
 - **Big-O (worst case),**
 - **Omega (best case), and**
 - **Theta (average case).**

1. Big O — $O(n)$

Meaning: Maximum time an algorithm can take.

Used for: Worst case

Example: In linear search, worst case is $O(n)$.

2. Omega — $\Omega(n)$

Meaning: Minimum time an algorithm can take.

Used for: Best case

Example: In linear search, if the element is first, it's $\Omega(1)$.

3. Theta — $\Theta(n)$

Meaning: Exact time (tight bound).

Used for: Average/typical case

Example: In linear search, Average case is $\Theta(n/2)$

Programs On Time Complexity:

```
#Example:1
n=5
for i in range(n):
    print("division-3")
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
division-3
division-3
division-3
division-3
division-3
# here, the for loop will run n=5 times and each time it runs ,it prints "division-3"
#the time taken is n times
# therefore time complexity is  $O(n)$  here
```

Example 2: Nested loops.

n=5

for i in range(n):# outer loop runs n times

for j in range(n): # everytime the outer loop runs the inner loop runs n times

print(i,end=" ")

#time taken is $n*n$ times

therefore time complexity is $O(n^2)$ here

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 4 4 4 4 4

Example:3

n=5

for i in range(n):

print(i,end=" ")

print("\n")

for j in range(n):

print(j,end=" ")

#here we have two separate loops

#the first for loop runs n times

then second for loop runs n times

therefore time taken is $n+n=2n$, time complexity is $O(n)$ here

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

0 1 2 3 4

0 1 2 3 4

Program On Space Complexity:

```
n=5
squares=[] #initially the list is empty
for i in range(1,n+1):
    squares.append(i*i) ## add square of i to the list, so extra space is required
print(squares) #list has n elements

# Extra space used is proportional to n → therefore space complexity is O(n)
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
[1, 4, 9, 16, 25]
```

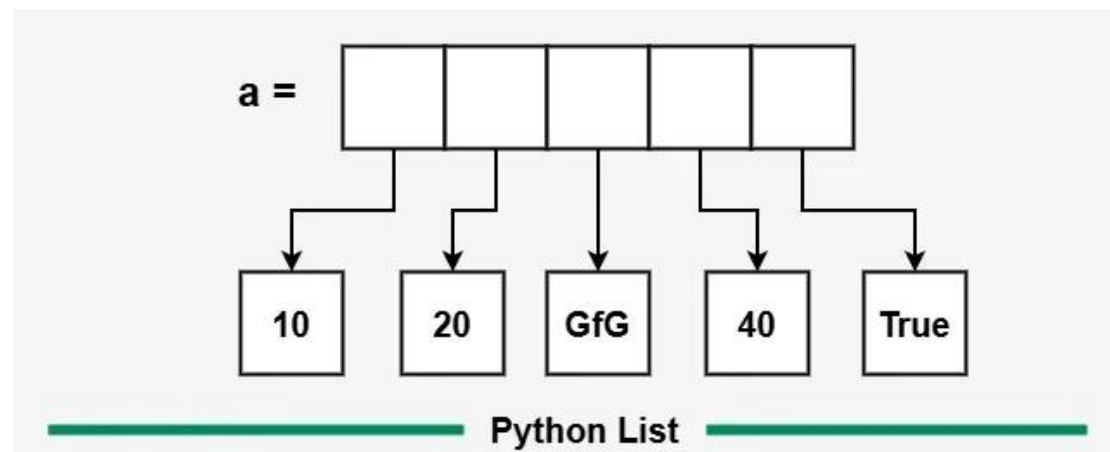
LIST DATA STRUCTURE IN PYTHON:

In Python, a list is a built-in dynamic sized array (automatically grows and shrinks).

It allows you to store an ordered collection of items.

- List can store different data types.
- List can contain duplicate items.
- List in Python are Mutable. Hence, we can modify, replace or delete the items.
- List are ordered. It maintain the order of elements based on how they are added.
- Accessing items in List can be done directly using their position (index), starting from 0.

Lists Store References, Not Values



- Each element in a list is not stored directly inside the list structure. Instead, the list stores references (pointers) to the actual objects in memory.
- The list itself is a container with references (addresses) to the actual values.
- Python internally creates separate objects for 10, 20, "GfG", 40 and True, then stores their memory addresses inside **a**.
- This means that modifying an element doesn't affect other elements but can affect the referenced object if it is mutable

LIST Basic Operations

Below are the LIST operations.

1. Create
2. Access
3. Add
4. Update
5. Remove/delete
6. Concatenation
7. Slicing

Creating a list:

A list can be created using square brackets [] or the list constructor list()

```
a=[10,20,30]
print(a)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[10, 20, 30]
```

Using list() Constructor

We can also create a list by passing an iterables (like a string, tuple or another list) to list() function.or convert other iterables to list

Syntax: list(iterable)

```
# passing a tuple and using list() method
a = list((1, 2, 3, 'apple', 4.5))
print(a)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[1, 2, 3, 'apple', 4.5]

#convert other iterables like string,tuple,set to list
str=list("hello")
tup=list((1,2,3))
set=list({1,2,3})
print(str)
print(tup)
print(set)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
['h', 'e', 'l', 'l', 'o']
[1, 2, 3]
[1, 2, 3]
```

Create empty list:

To create an empty list use either list() constructor method or use square brackets.

```
# empty list using list() method
a= list()
print(a)
b=[]# empty list using [] square brackets
print(b)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[]
[]
```


different data types are also allowed

```
print("list of integers")
a=[10,20,30]
print(a)
print("list of floating point numbers")
b=[0.1,0.2,0.3]
print(b)
print("list of strings")
c=['apple','banana','mango']
print(c)
#storing different types of data in a single list which is not possible in arrays.
d=[1,5.6,'div-3','div-6',[1,2,3],(5,6,7),{'a','b','c'}]
print(d)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
list of integers
[10, 20, 30]
list of floating point numbers
[0.1, 0.2, 0.3]
list of strings
['apple', 'banana', 'mango']
[1, 5.6, 'div-3', 'div-6', [1, 2, 3], (5, 6, 7), {'c', 'b', 'a'}]
```

Check the data type of the list:

```
a=[10,20,30]
print(a)
print(type(a))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[10, 20, 30]
<class 'list'>
```

Check the data type of individual elements:

```
a=[1,5.6,'div-3','div-6',[1,2,3],(5,6,7),{'a','b','c'}]
print(type(a[0]))
print(type(a[1]))
print(type(a[3]))
print(type(a[4]))
print(type(a[5]))
print(type(a[6]))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'set'>
```

Accessing the elements using Negative and positive indexing:

Elements in a list can be accessed using **indexing**. Python indexes start at **0**, so **a[0]** will access the first element, while **negative indexing** allows us to access elements from the end of the list. Like index -1 represents the last elements of list.

```
print("list of integers")
a=[10,20,30]
print(a)
print("list of floating point numbers")
b=[0.1,0.2,0.3]
print(b)
print("list of strings")
c=['apple','banana','mango']
print©
```

```
print("Accessing the elements using positive index")
a=[10,20,30]
print(a[0])
print(a[1])
print(a[2])
print("Accessing the elements using negative index")
a=[10,20,30]
print(a[-1])
print(a[-2])
print(a[-3])

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Accessing the elements using positive index
10
20
30
Accessing the elements using negative index
30
20
10
```

Creating List with Repeated Elements

We can create a list with repeated elements using the multiplication operator.

```
# Create a list [12, 12, 12, 12, 12]
a = [12] * 5
# Create a list [0, 0, 0, 0, 0]
b = [0] * 5
print(a)
print(b)
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
[12, 12, 12, 12, 12]  
[0, 0, 0, 0, 0]
```

Methods In Python

In Python, a **method** is: Methods are like functions, but they are called using dot (.) notation on objects (like lists, strings, dictionaries, etc.).

Methods can change the object or return some result based on the object.

```
name = "python"  
print(name.upper()) # upper() is a method that returns 'PYTHON'  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
PYTHON
```

Here, upper() is a method of the string object name. Like wise we have list methods

Python List methods

methods are built-in functions that allow us to perform various operations on lists, such as adding, removing, or modifying elements.

List Methods

Let's look at different list methods in Python:

Method	Description
--------	-------------

<u>append()</u>	Adds an element at the end of the list.Syntax: list_name.append(element)
---------------------------------	--

<u>clear()</u>	Removes all the elements from the list.Syntax: list_name.clear()
--------------------------------	--

<u>copy()</u>	Returns a copy of the list.Syntax: list_name.copy()
-------------------------------	---

<u>count()</u>	Returns the number of elements with the specified value. Syntax: list_name.count(element)
--------------------------------	--

[extend\(\)](#) Add the elements of a list (or any iterable), to the end of the current list.
Syntax: list_name.extend(iterable)

[index\(\)](#) Returns the index of the first element with the specified value.
Syntax: list_name.index(element)

[insert\(\)](#) Adds an element at the specified position.Syntax: list_name.insert(index, element)

[pop\(\)](#) Removes the element at the specified position.Syntax: list_name.pop(index)

[remove\(\)](#) Removes the first item with the specified value.Syntax:
list_name.remove(element)

[reverse\(\)](#) Reverses the order of the list.Syntax: list_name.reverse()

[sort\(\)](#) Sorts the list in ascending order (by default).Syntax: list_name.sort(key=None, reverse=False) or list_name.sort(key=None, reverse=True) for descending order.

Examples On Some Of The Methods

```
#Index Method
a = ['apple', 'banana', 'cherry', 'banana']
print(a.index('banana')) # Output: 1

#reverse method
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers) # Output: [5, 4, 3, 2, 1]

# sort method
marks = [90, 40, 60, 80, 70]
marks.sort()
print(marks) # Output: [40, 60, 70, 80, 90]
marks.sort(reverse=True) #If you want descending order,
print(marks) # Output: [90, 80, 70, 60, 40]
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
1
[5, 4, 3, 2, 1]
[40, 60, 70, 80, 90]
[90, 80, 70, 60, 40]
```

Adding Elements into List

We can add elements to a list using the following methods:

append(): Adds an element at the end of the list.

extend(): Adds multiple elements to the end of the list.

insert(): Adds an element at a specific position.

```
# Initialize an empty list
a = []
# Adding 10 to end of list
a.append(10)
print("After append(10) the list is:", a)
# Inserting 5 at index 0
a.insert(0, 5)
print("After insert(0, 5):", a)
# Adding multiple elements [15, 20, 25] at the end
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
After append(10) the list is: [10]
After insert(0, 5): [5, 10]
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
```

Updating Elements into List

We can change the value of an element by accessing it using its index.

```
a = [10, 20, 30, 40, 50]
print("list before updating:")
print(a)
# Change the second element
a[1] = 25
print("list after updating:")
print(a)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
list before updating:
[10, 20, 30, 40, 50]
list after updating:
[10, 25, 30, 40, 50]
```

Removing Elements from List

We can remove elements from a list using:

remove(): Removes the first occurrence of an element.

pop(): Removes the element at a specific index or the last element if no index is specified.

del statement: Deletes an element at a specified index.or it can delete an entire list

Clear(): Empties the list

```
a = [10, 20, 30, 40, 50]
print(a) # Original list
a.remove(30) # Remove 30
print(a) # List after removing 30
popped_val = a.pop(1) # Remove element at index 1
print(popped_val) # Popped value
print(a) # List after pop
del a[0] # Delete element at index 0
print(a) # List after del
```

```
a.clear() # Clear all elements
print(a) # List after clear
del a # Delete entire list
print(a) # Error: 'a' no longer exists
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[10, 20, 30, 40, 50]
[10, 20, 40, 50]
20
[10, 40, 50]
[40, 50]
[]
Traceback (most recent call last):
  File "f.py", line 18, in <module>
    print(a) # Error: 'a' no longer exists
NameError: name 'a' is not defined
```

Iterating Over Lists

We can iterate the Lists easily by using a for loop or other iteration methods. Iterating over lists is useful when we want to do some operation on each item or access specific items based on certain conditions.

Using for Loop

```
a = ['apple', 'banana', 'cherry']
# Iterating over the list
for item in a:
    print(item)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
apple
banana
cherry
```


Nested Lists in Python

A nested list is a list within another list, which is useful for representing matrices or tables. We can access nested elements by chaining indexes.

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9] ]  
  
print(matrix[1]) #at index 1 [4,5,6] is present  
# Access element at row 2, column 3  
print(matrix[1][2])  
  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
[4, 5, 6]  
6
```

Concatenation Of List: Combining Two Or More Lists

```
#concatenation of two lists  
l1= [1, 2, 3]  
l2= [4, 5, 6]  
print(l1+l2)  
  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
[1, 2, 3, 4, 5, 6]
```

SLICING:

In Python, **slicing** means:

Extracting a part (subsection) of a sequence like a list, string, or tuple by specifying a range of indices.

You use **colon : operator** inside square brackets [] to do slicing.

Basic Syntax:

`sequence[start : end : step]`

In All The Sequences (List,Tuples,Strings)Slicing Works The Same Way

Parameters:

start (optional): Index to begin the slice (inclusive). Defaults to 0 if omitted.

end (optional): Index to end the slice (exclusive). Defaults to the length of list if omitted.

step (optional): Step size, specifying the interval between elements. Defaults to 1 if omitted

Python List Slicing

Python **list slicing** is fundamental concept that let us easily access specific elements in a list. We can use both positive and negative indexing for slicing.

Python List Slicing Syntax

`list_name[start : end : step]`

Get All The Items From a List

To retrieve all items from a list, we can use slicing without specifying any parameters.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get all elements in the list
print(a[:])
print(a[:])
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Explanation: Using `[:]` & `::]` without specifying any start, end, or step returns all elements of the list.

Get All Items Before/After a Specific Position

To get all the items from a specific position to the **end** of the list, we can specify the **start** index and leave the end blank.

And to get all the items before a specific index, we can specify the **end** index while leaving **start** blank.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get elements starting from index 2 to the end of the list
b = a[2:]
print(b)
# Get elements starting from index 0 to index 3 (excluding 3th index)
c = a[:3]
print(c)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[3, 4, 5, 6, 7, 8, 9]
[1, 2, 3]
```

Get All Items Between Two Positions

To extract elements between two specific positions, specify both the start and end indices

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get elements from index 1
# to index 4 (excluding index 4)
b = a[1:4]
print(b)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[2, 3, 4]
```

Get Items At Specified Intervals

To extract elements at specific intervals, use the **step** parameter.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get every second element from the list
# starting from the beginning
b = a[::2]
print(b)
# Get every third element from the list
# starting from index 1 to 8(exclusive)
c = a[1:8:3]
print(c)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[1, 3, 5, 7, 9]
[2, 5, 8]
```

Negative Indexing

Negative indexing is useful for **accessing elements from the end of the list**. The last element has an index of **-1**, the second last element **-2**, and so on.

Negative indexing makes it easy to get items without needing to know the exact length of the list.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get elements starting from index -2
# to end of list
b = a[-2:]
print(b)
# Get elements starting from index 0
# to index -3 (excluding 3th last index)
c = a[:-3]
print(c)
# Get elements from index -4
```

```
# to -1 (excluding index -1)
d = a[-4:-1]
print(d)

# Get every 2nd elements from index -8
# to -1 (excluding index -1)
e = a[-8:-1:2]
print(e)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[8, 9]
[1, 2, 3, 4, 5, 6]
[6, 7, 8]
[2, 4, 6, 8]
```

Length Of The List:

Use len() function

```
a=[1,2,3,4]
print(len(a))

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
4
```

TUPLES DATA STRUCTURE IN PYTHON:

A tuple in [Python](#) is an immutable ordered collection of elements. Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable). Tuples can hold elements of different data types. The main characteristics of tuples are being ordered, heterogeneous and immutable.

Cannot update/append/extend/remove items from the tuple once it is created.

tuples Basic Operations

Below are the tuple operations.

- 1) Create
- 2) Access
- 3) Add
- 4) Update
- 5) Remove/delete
- 6) Concatenation
- 7) Slicing

Creating a Tuple

A tuple is created by placing all the items inside parentheses (), separated by commas. A tuple can have any number of items and they can be of different [data types](#).

```
#create using () round brackets
a=(1,2,3,4,5)
print(a)
print(type(a))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(1, 2, 3, 4, 5)
<class 'tuple'>
```

Tuple Constructor:

With tuple constructor method you can convert other data types like list,string,sets to tuples or you can create an empty tuple .

```
a=[1,2,3]#a is a list
t=tuple(a)# converts a to tuple
print(a)
print(type(a))
t1=tuple([4,5,6])#list to tuple
t2=tuple("hello")#string to tuple
print(t1)
print(t2)
[1, 2, 3]
<class 'list'>
(4, 5, 6)
('h', 'e', 'l', 'l', 'o')
```

Create Empty Tuple:

```
#empty tuple
a=()
print(a)
print(type(a))
# empty tuple using tuple constructor method
a=tuple()
print(a)
print(type(a))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
()
<class 'tuple'>
()
<class 'tuple'>
```

Create a Tuple With Single Element:

Single element tuple must include a comma otherwise, it is treated as a normal data type

```
a=(5,)
print(a)
print(type(a))#here a is tuple as it is using comma
b=(5)
print(b)
print(type(b))#here b is an integer not a tuple
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(5,)
<class 'tuple'>
5
<class 'int'>
```

Concatenation

```
t1=(1,2,3)
t2=(4,5)
t3=(t1+t2)
print(t3)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(1, 2, 3, 4, 5)
```

Repetition:

```
t1=(1,2,3)*3
print(t1)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```


Membership test:

```
t1=(10,20,30,40,50)
print(20 in t1)
print(65 in t1)
print(75 not in t1)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
True
False
True
PS C:\U
```

Accessing The Elements Using Negative And Positive Indexing:

Elements in a tuple can be accessed using **indexing**. Python indexes start at **0**, so **a[0]** will access the first element, while **negative indexing** allows us to access elements from the end of the tuple. Like index -1 represents the last elements of tuple.

```
print("Accessing the elements using positive index")
a=(10,20,30)
print(a[0])
print(a[1])
print(a[2])
print("Accessing the elements using negative index")
a=(10,20,30)
print(a[-1])
print(a[-2])
print(a[-3])
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Accessing the elements using positive index
10
20
30
```

Accessing the elements using negative index

30

20

10

Updating Tuples : Not Possible

```
t1=(10,20,30,40,50)
```

```
t1[2]=95#not possible because tuples are immutable
```

```
print(t1)
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
Traceback (most recent call last):
```

```
File "f.py", line 2, in <module>
```

```
    t1[2]=95
```

```
TypeError: 'tuple' object does not support item assignment
```

Deleting a Tuple:

removing individual element from the tuple is not possible, but we can delete the whole tuple using del keyword

```
t1=(10,20,30,40,50)
```

```
del t1
```

```
print(t1)#error,since t1 is deleted
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
Traceback (most recent call last):
```

```
File "f.py", line 3, in <module>
```

```
    print(t1)#error,since t1 is deleted
```

```
NameError: name 't1' is not defined
```

Nested Tuple:

```
t1=(1,2,3,4)
t2=('hello','div-3')
t3=(t1,t2)
print(t3)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
((1, 2, 3, 4), ('hello', 'div-3'))
```

Length Of The Tuple:

Use len() function find the number of elements present in the tuple

```
t1=(1,2,3,4)
print(len(t1))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
4
```

Tuple Packing:

Tuple packing is the process of assigning multiple values to a single tuple variable

```
t=(1,2,3,'python') #here 1,2,3,'python' are packed into the tuple t
print(t)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
(1, 2, 3, 'python')
```

Tuple Unpacking:

Tuple packing is the process of extracting values from a tuple into separate variables

```
t=(1,2,3,'python') #here 1,2,3,'python' are packed into the tuple t
a,b,c,d=t #tuple unpacking
# here each variable on LHS gets corresponding value from the tuple t
print(a)
print(b)
print(c)
print(d)
```

```
# or
a,b,c,d=1,2,3,'python'
print(a)
print(b)
print(c)
print(d)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
1
2
3
python
1
2
3
python
```

Tuple Methods

Python has two built-in methods that you can use on tuples.

Method	Description
<u>count()</u>	Returns the number of times a specified value occurs in a tuple Syntax: tuple.count(element)
<u>index()</u>	Searches the tuple for a specified value and returns the position of where it was found Syntax: tuple.index(element, start, end)

```
t=(1,2,3,4,5,2,67,25,4,4,19,4)
print(t.count(4))#no of times item 4 occurs
print(t.index(4))# the index of 4 is 3 when it occurred for the first time
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
4
3
```

Tuple Slicing:

Same As Lists Check List Slicing Concept Above.

```
# Creating a tuple
t = (10, 20, 30, 40, 50, 60, 70, 80)
# Example 1: Slicing from index 2 to 5 (excluding 5)
s1 = t[2:5]
print("Slice 1:", s1) # Output: (30, 40, 50)
# Example 2: Slicing from the start to index 4 (excluding 4)
s2 = t[:4]
print("Slice 2:", s2) # Output: (10, 20, 30, 40)
# Example 3: Slicing from index 3 to the end
s3 = t[3:]
print("Slice 3:", s3) # Output: (40, 50, 60, 70, 80)
# Example 4: Slicing every second element from index 1 to 6 (excluding 6)
s4 = t[1:6:2]
print("Slice 4:", s4) # Output: (20, 40, 60)
# Example 5: Slicing the entire tuple
s5 = t[:]
print("Slice 5:", s5) # Output: (10, 20, 30, 40, 50, 60, 70, 80)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Slice 1: (30, 40, 50)
Slice 2: (10, 20, 30, 40)
Slice 3: (40, 50, 60, 70, 80)
Slice 4: (20, 40, 60)
Slice 5: (10, 20, 30, 40, 50, 60, 70, 80)
```

SETS IN DATA STRUCTURE IN PYTHON:

A Set in Python is used to store **unordered** collection of **unique items** with the following properties.

- **No duplicate elements.** If we try to insert the same item again, it overwrites previous one.
- **An unordered collection.** When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- **Mutable**, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

```
s = {10, 50, 20}
print(s)
print(type(s))#Note : There is no specific order for set elements to be printed
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{10, 50, 20}
<class 'set'>
```

Type Casting With Python Set Method

The Python set() method is used for type casting.

```
# typecasting list to set
s = set(["a", "b", "c"])
print(s)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{'b', 'a', 'c'}
```

Check Unique And Immutable With Python Set

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

```
# Python program to demonstrate that a set cannot have duplicate values  
# and we cannot change its items
```

```
# a set cannot have duplicate values
```

```
s = {"DIV-3", "DIV-6", "DIV-3"}
```

```
print(s)
```

```
# values of a set cannot be changed after creation
```

```
s[1] = "division-6"
```

```
print(s)
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
{'DIV-6', 'DIV-3'}
```

```
Traceback (most recent call last):
```

```
File "f.py", line 10, in <module>
```

```
s[1] = "division-6"
```

```
TypeError: 'set' object does not support item assignment
```

Heterogeneous Element With Python Set

Python sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc data types.

```
# Python example demonstrate that a set
```

```
# can store heterogeneous elements
```

```
s = { 17, 3.6, 'python', True}
```

```
print(s)
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
{3.6, 17, 'python', True}
```

Empty Set:

Can be created using set constructor method:

```
a=set()# since a={} is considered as an empty dictionary not an empty set.  
print(type(a))  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
<class 'set'>
```

Adding Or Removing Elements Using Methods

```
s=set()  
s.add(10)  
s.add(20)  
s.add(30)  
s.add(40)  
print(s)  
s.remove(30)  
s.remove(45)#error since 65 is not present in the set  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
{40, 10, 20, 30}  
Traceback (most recent call last):  
  File "f.py", line 8, in <module>  
    s.remove(45)#error since 65 is not present in the set  
KeyError: 45
```

A set cannot have list,dictionaries,set itself as an item in the set.

Since they are mutable and a set item are immutable.

```
s=set()  
s.add([1,2,3])#error since set cannot have lists as an item since lists are mutable.  
print(s)  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
Traceback (most recent call last):  
  File "f.py", line 2, in <module>  
    s.add([1,2,3])#error since set cannot have lists as an item since lists are mutable.  
TypeError: unhashable type: 'list'
```


Adding Set And Dictionaries Or List Is Not Possible

```
s=set()
s.add({1:"a"})#error
s.add({1,2,3})#error
```

Using pop() Method

cannot pass item name/index since sets are unordered therefor items cannot be accessed using indexing.

```
s={1,2,3,4,5}
print(s)
s.pop()#randomly pops any element
s.pop(2)#error
Pop(-1)#error
```

Using clear() Method

```
s={1,2,3,4,5}
print(s)
s.clear()#empties the set
print(s)
```

Union Operation On Python Sets:

Two sets can be merged using union() function or | operator.
the duplicates are removed.

Intersection Operation On Python Sets:

This can be done through intersection() or & operator. Common Elements are selected.

```
s1={1,2,3,4}
s2={3,4,5,6}
print(s1.union(s2))#unique elements
print(s1.intersection(s2))#common elements
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{1, 2, 3, 4, 5, 6}
{3, 4}
s1={1,2,3,4}
s2={3,4,5,6}
print(s1|s2)#unique elements
print(s1&s2)#common elements
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{1, 2, 3, 4, 5, 6}
{3, 4}
```

Membership Test:

```
s1={1,2,3,4}
print(1 in s1)
print(3 not in s1)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
True
False
```

FROZENSET

A frozenset is just like a set, but **immutable** — once you create it, you cannot change it (you can't add, remove, or update elements).

Think of it as a "frozen" set: fixed, unchangeable.

So that it can be used as keys in dictionaries or as elements of other sets.

```
a={10,20,30,40}
b=frozenset(a)
print(a)#a is a normal se
print(b)#b is a frozenset of a
b.add(25)#error
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{40, 10, 20, 30}
frozenset({40, 10, 20, 30})
Traceback (most recent call last):
  File "f.py", line 5, in <module>
    b.add(25)#error
AttributeError: 'frozenset' object has no attribute 'add'
```

You can do all read-only operations, like:

Union: combine sets

Intersection: common elements

Difference: elements in one but not the other

Subset, Superset checks

What You Cannot Do With Frozenset

You can't do these:

b.add(5)

b.remove(2)

b.clear()

b.update([...])

Since b is frozen.

DICTIONARY DATA STRUCTURE IN PYTHON

A **Python dictionary** is a data structure that stores the value in **key: value** pairs.

Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

How to Create a Dictionary

In Python, a dictionary can be created by placing a sequence of elements within curly {} braces, separated by a 'comma'.

```
# create dictionary using { }  
d1 = {"name": 'john', "Rollno": 1234, "marks": 98.5}  
print(d1)  
  
# create dictionary using dict() constructor  
d2 = dict(a = "apple", b = "banana", c = "mango")  
print(d2)  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
{'name': 'john', 'Rollno': 1234, 'marks': 98.5}  
{'a': 'apple', 'b': 'banana', 'c': 'mango'}
```

PROPERTIES OF DICTIONARY

- From Python 3.7 Version onward, Python dictionary are **Ordered**.
- **Dictionary keys are case sensitive:** the same name but different cases of Key will be treated distinctly.
- **Keys must be immutable:** This means keys can be strings, numbers, or tuples but not lists.
- **Keys must be unique:** Duplicate keys are not allowed and any duplicate key will overwrite the previous value.

Accessing Dictionary Items

We can access a value from a dictionary by using the **key** within square brackets `or get()` method.

```
# create dictionary using { }
d= {"name": 'john', "Rollno": 1234, "marks": 98.5}
print(d)

# Access using key
print(d["name"])

# Access using get(): pass key as an argument
print(d.get("rollno"))
```

Adding and Updating Dictionary Items

We can add new key-value pairs or update existing keys by using assignment

```
# create dictionary using { }
d= {"name": 'john', "Rollno": 1234, "marks": 98.5}
print(d)

# Adding a new key-value pair
d["section"] = "DIVISION-6"

# Updating an existing value
d["marks"] = 95.2 # pass the name of the key to update its value
print(d)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{'name': 'john', 'Rollno': 1234, 'marks': 98.5}
{'name': 'john', 'Rollno': 1234, 'marks': 95.2, 'section': 'DIVISION-6'}
```

Update Using .update() Method

Change the values associated with the keys using update() method.

```
d={
    "name": "John",
    "rollno": 3456,
    "marks": 90
}
d.update({"rollno":1234})
print(d)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{'name': 'John', 'rollno': 1234, 'marks': 90}
```

Removing Dictionary Items

We can remove items from dictionary using the following methods:

- **del**: Removes an item by key.
- **pop()**: Removes an item by key and returns its value.
- **clear()**: Empties the dictionary.
- **popitem()**: Removes and returns the last key-value pair.

```
# create dictionary using { }
d= {
    "name": 'john',
    "Rollno": 1234,
    "marks": 98.5,
    "section":"DIVISION-3",
    "age":18
}
print("original dictionary:",d)
# Using del to remove an item
del d["age"]
print(d)
```

```
# Using pop() to remove an item and return the value
val = d.pop("section")
print(f'using pop: {val}')

# Using popitem to removes and return the last key-value pair.
key, val = d.popitem()
print(f'Key: {key}, Value: {val}')

# Clear all items from the dictionary
d.clear()
print(d)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

original dictionary: {'name': 'john', 'Rollno': 1234, 'marks': 98.5, 'section': 'DIVISION-3', 'age': 18}

{'name': 'john', 'Rollno': 1234, 'marks': 98.5, 'section': 'DIVISION-3'}

using pop:DIVISION-3

Key: marks, Value: 98.5

{}

```

Iterating Through a Dictionary

We can iterate over **keys** [using [keys\(\) method](#)] , **values** [using [values\(\) method](#)] or both [using [item\(\) method](#)] with a [for loop](#).

Iterate over keys:

```
# create dictionary using { }
d= {
    "name": 'john',
    "Rollno": 1234,
    "marks": 98.5,
}

# method-1 :Iterate over keys
for key in d:
    print(key)
print("\n")

# method-2 :Iterate over keys using .keys() method
for key in d.keys():
    print(key)
```

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

name
Rollno
Marks

name
Rollno
marks

Iterate Over Values:

```
# create dictionary using { }
d= {
    "name": 'john',
    "Rollno": 1234,
    "marks": 98.5,

}

# method-1 :Iterate over values
for x in d:
    print(d[x])
print("\n")

# method-2 :Iterate over values using .values() method
for value in d.values():
    print(value)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
john
john
1234
98.5

john
1234
98.5
```

Iterate Over Key Value Pairs:

```
# create dictionary using { }  
d= {  
    "name": 'john',  
    "Rollno": 1234,  
    "marks": 98.5,  
  
    }  
  
# Iterate over key-value pairs using .items() method  
for key, value in d.items():  
    print(f"{key}: {value}")  
  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
name: john  
Rollno: 1234  
marks: 98.5
```

Nested Dictionary

A dictionary inside another dictionary is called a nested dictionary.

It looks like this:

```
students = {  
    "student1": {  
        "name": "John",  
        "age": 18,  
        "marks": 90  
    },  
    "student2": {  
        "name": "Mike",  
        "age": 19,  
        "marks": 95  
    },  
    "student3": {  
        "name": "will",  
        "age": 17,  
        "marks": 65  
    }  
}  
  
#access the entire dictionary dictionary  
print(students)  
  
#access values inside a nested dictionary  
print(students["student1"]["name"]) # Output: John  
print(students["student2"]["marks"]) # Output: 95  
  
#iterate over a nested dictionary  
for key, val in students.items():  
    print(f"Details of {key}:")  
    for ikey, ival in val.items():  
        print(ikey, ":", ival)  
    print() # for new line
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py

{'student1': {'name': 'John', 'age': 18, 'marks': 90}, 'student2': {'name': 'Mike', 'age': 19, 'marks': 95}, 'student3': {'name': 'will', 'age': 17, 'marks': 65}}
```

John
95

Details of student1:
name : John
age : 18
marks : 90

Details of student2:
name : Mike
age : 19
marks : 95

Details of student3:
name : will
age : 17
marks : 65

Method Copy()

```
d={
    "name": "John",
    "rollno": 3456,
    "marks": 90
}
a=d.copy()
print(d)# a is the copy of d now with copy() method
b=dict(d)
print(b)#b is also the copy of d with dict() method
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
{'name': 'John', 'rollno': 3456, 'marks': 90}
{'name': 'John', 'rollno': 3456, 'marks': 90}
```

Strings In Python:

A string is a sequence of characters. Python treats anything inside quotes as a string. This includes letters, numbers, and symbols. Python has no character data type so single character is a string of length 1.

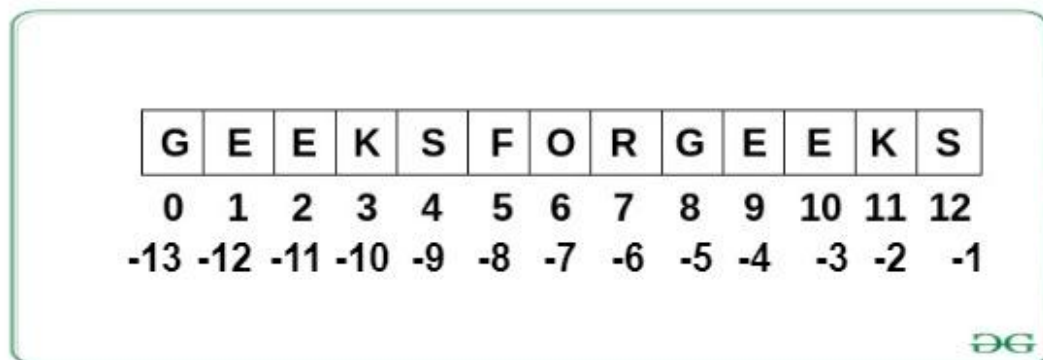
Creating a String

Strings can be created using either **single** (') , **double** (") quotes or we can use **triple quotes** (""" or """).

```
s1 = 'Masrath'
s2 = "Parveen"
s3="triple quotes"
print(s1)
print(s2)
print(s3)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Masrath
Parveen
triple quotes
```

Accessing Characters In Python String

Strings in Python are sequences of characters, so we can access individual characters using **indexing**. Strings are indexed **starting** from **0** and **-1** from **end**. This allows us to retrieve specific characters from the string.



```
s = "GeeksforGeeks"

# Accesses first character: 'G'
print(s[0])

# Accesses 5th character: 's'
print(s[4])

# using negative indexing
# Accesses 3rd character: 'k'
print(s[-10])

# Accesses 5th character from end: 'G'
print(s[-5])

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
G
s
k
G
```

String Slicing

Slicing is a way to extract portion of a string by specifying the **start** and **end** indexes. The syntax for slicing is **string[start:end:step]**, where **start** starting index and **end** is stopping index (**excluded**) and step is increment/decrement the **step** value

SAME AS LIST

```
s = "thisisdivision-3"

# Retrieves characters from index 1 to 3:
print(s[1:4])

# Retrieves characters from beginning to index 2:
print(s[:3])

# Retrieves characters from index 3 to the end:
print(s[3:])
```

String Immutability

Strings in Python are immutable. This means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like **concatenation, slicing, or formatting** to create new strings based on the original.

```
s = "division-6"

# Trying to change the first character raises an error
# s[0] = 'D' # Uncommenting this line will cause a TypeError
# Instead, create a new string
s = "D" + s[1:]
print(s)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Division-6
```

Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

```
s = "division-6"
# Deletes entire string
del s
print(s) #error .since s is deleted now
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Traceback (most recent call last):
  File "f.py", line 5, in <module>
    print(s) #error .since s is deleted now
NameError: name 's' is not defined
```


Concatenating and Repeating Strings

We can concatenate strings using + **operator** and repeat them using * **operator**.

```
s1 = "Hello"
s2 = "World"
s3 = s1 + " " + s2 #concatenation
print(s3)
print(s1*3)#repetition
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Hello World
HelloHelloHello
```

Using *in* for String Membership Testing

The in /not in keyword checks if a particular substring is **present/ not present** in a string.

```
s1 = "Division-3"
print('D' in s1)
print('z' not in s1)
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
True
True
```

STRING METHODS

Method	Description	Example Code & Output
lower()	Converts the string to lowercase .	s = "HELLO" print(s.lower()) Output: 'hello'
upper()	Converts the string to uppercase .	s = "hello" print(s.upper()) Output: 'HELLO'
strip()	Removes leading and trailing	s = " hello "

Method	Description	Example Code & Output
	spaces.	print(s.strip()) Output: 'hello'
replace()	Replaces substring with another.	s = "hello world" print(s.replace("world", "Python")) Output: 'hello Python'
split()	Splits the string into a list of substrings.	s = "apple,banana,grape" print(s.split(",")) Output: ['apple', 'banana', 'grape']
Len()	Finds the length of the string	s="apple" print(len(s))

DIFFERENCE BETWEEN LIST,TUPLE,SET AND DICTIONARY				
Parameters	List	Tuple	Set	Dictionary
Definition	A list is an ordered, mutable collection of elements.	A tuple is an ordered, immutable collection of elements.	A set is an unordered collection of unique elements.	A dictionary is an unordered collection of key-value pairs.
Syntax	Syntax includes square brackets [,] with ',' separated data.	Syntax includes curved brackets (,) with ',' separated data.	Syntax includes curly brackets { , } with ',' separated data.	Syntax includes curly brackets { , } with ',' separated key-value data.
Creation	A list can be created using the list() function or simple assignment to [].	Tuple can be created using the tuple() function.	A set dictionary can be created using the set() function.	A dictionary can be created using the dict() function.
Empty Data Structure	An empty list can be created by l = [].	An empty tuple can be created by t = ().	An empty set can be created by s = set().	An empty dictionary can be created by {}.
Order	It is an ordered collection of data.	It is also an ordered collection of data.	It is an unordered collection of data.	Ordered collection in Python version 3.7, unordered in Python Version=3.6.
Duplicate Data	Duplicate data entry is allowed in a List.	Duplicate data entry is allowed in a Tuple.	All elements are unique in a Set.	Keys are unique, but two different keys CAN have the same value.
Indexing	Has integer based indexing that starts from '0'.	Also has integer based indexing that starts from '0'.	Does NOT have an index based mechanism.	Has a Key based indexing i.e. keys identify the value.

Addition	New items can be added using the append() method.	Being immutable, new data cannot be added to it.	The add() method adds an element to a set.	update() method updates specific key-value pair.
Deletion	Pop() method allows deleting an element.	Being immutable, no data can be popped/deleted.	Elements can be randomly deleted using pop().	pop(key) removes specified key along with its value.
Sorting	sort() method sorts the elements.	Immutable, so sorting method is not applicable.	Unordered, so sorting is not advised.	Keys are sorted by using the sorted() method.
Search	index() returns index of first occurrence.	index() returns index of first occurrence.	Unordered, so searching is not applicable.	get(key) returns value against specified key.
Reversing	reverse() method reverses the list.	Immutable, so reverse method is not applicable.	Unordered, so reverse is not advised.	No integer-based indexing, so no reversal.
Count	count() method returns occurrence count.	count() method returns occurrence count.	count() not defined for sets.	count() not defined for dictionaries.

Difference Between Mutable And Immutable Data Types

Aspect	Mutable Data Types	Immutable Data Types
Meaning	Data types that can be changed after creation.	Data types that cannot be changed once created.
Memory Address	Can stay the same even if content is modified.	Any change creates a new object with a different memory address.
Examples	List, Dictionary, Set, bytearray	String, Tuple, Integer, Float, Boolean, bytes
Behavior	Elements inside can be updated, added, removed .	Entire object stays constant ; if you modify, a new object is created.
Use Case	Useful when you need to modify data frequently .	Useful when you want constant, safe data.

UNIT-II PART-2

SEARCHING AND SORTING

SEARCHING

Searching is one of the most common operations performed in an array or a list. searching can be defined as the operation of finding a particular element or a group of elements in the list.

There are several searching algorithms. The most commonly used among them are:

- **Linear Search**
- **Binary Search**

Linear Search (Sequential Search)

Definition: In a linear search, each element in the array or list is checked sequentially until the desired element is found or the end is reached.

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element or group of elements is found. Otherwise, the search continues till the end of the data set. This has a time complexity of $O(N)$ where 'N' is the length of the array

Example:

looking for a name in a random attendance list-you check the names one by one from top to bottom-that's linear search

When to use: Linear search is mainly used to find an element in an unordered list.

Best for: Small or unsorted datasets.

Algorithm for Linear Search

Input:

- A list `arr[]` containing elements.
- An integer `n` representing the number of elements ($n = \text{len}(\text{arr})$).
- A search key `key` to be found in the list.

Start:

1. Set an index variable, starting at 0 (i.e., `arr[0]`).
2. Traverse the list:
3. For each index `i` from 0 to `n-1`:
4. Compare `arr[i]` with `key`.
5. If match is found:
 - a) If `arr[i] == key`, then:
 - b) Return the index `i`.
6. If end of list is reached without a match:
 - a) After checking all elements (`i` from 0 to `n-1`):
 - b) If no element matches `key`, return -1.

Output:

If the key is found, output the index.

If not found, output -1.

ADVANTAGES:

Works on any data (no pre-sorting required).

Easy to implement.

DISADVANTAGES:

Slow for large datasets because every element might need to be checked.

Working of Linear search

Now, let's see the working of the linear search Algorithm.

To understand the working of linear search algorithm, let's take an unsorted array. It will be easy to understand the working of linear search with an example.

Let the elements of array are -

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

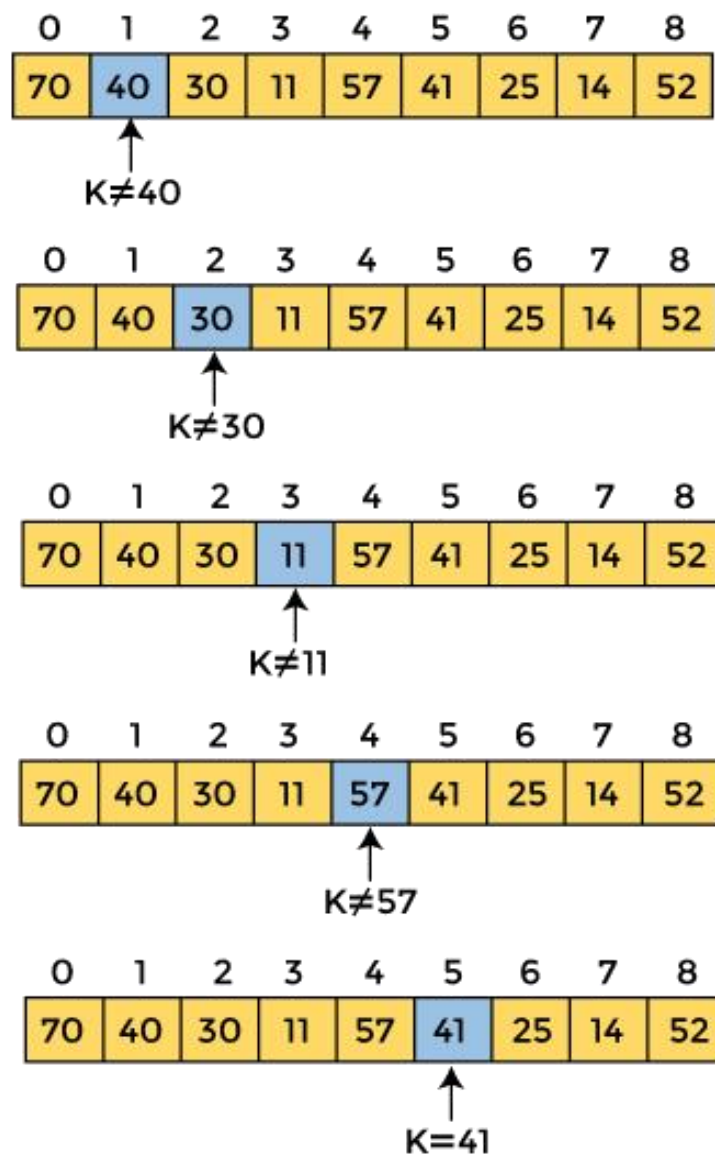
Let the element to be searched is $K = 41$

Now, start from the first element and compare K with each element of the array.

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
 $K \neq 70$

The value of K , i.e., **41**, is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.



Now, the element to be searched is found. So algorithm will return the index of the element match.

Time Complexity analysis of linear search:

Best Case Complexity - In Linear search, best case occurs when the element we are finding is at the first position of the list. The best-case time complexity of linear search is $O(1)$.

Average Case Complexity - The average case time complexity of linear search is $O(n)$.

Worst Case Complexity - In Linear search, the worst case occurs when the element we are looking is present at the end of the list. The worst-case in linear search could be when the target element is not present in the given list, and we have to traverse the entire list. **The worst-case time complexity of linear search is $O(n)$.**

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(n)$
Worst Case	$O(n)$

SPACE COMPLEXITY ANALYSIS

the space complexity of Linear Search is $O(1)$.

Because linear search only needs a few extra variables (like an index, or a flag to say "found" or "not found"), no matter how big the input list/array is.

It does not create any new data structures that grow with input size.

How to Implement Linear Search Algorithm?

The Linear Search Algorithm can be implemented in the following two ways

- Iterative Linear Search Algorithm
- Recursive Linear Search Algorithm

Iterative Linear Search Algorithm

```
# write a program/function to implement linear search(using Iteration)
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

arr = [5, 8, 2, 9, 1]
target = 9
result = linear_search(arr, target)
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Element found at index 3
```

Recursive Linear Search Algorithm

#write a program to implement linear search using recursion

```
def linerec(arr, key, index=0):  
    if index >= len(arr):  
        return -1  
    if arr[index] == key:  
        return index  
    return linerec(arr, key, index + 1) # recursion
```

```
arr = [5, 8, 2, 9, 1]
```

```
key = 9
```

```
result = linerec(arr, key) #function call
```

```
if result != -1:
```

```
    print(f"Element found at index {result}")
```

```
else:
```

```
    print("Element not found")
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
Element found at index 3
```

Importance Of Time Complexity Analysis In Algorithm Design.

Aspect	Importance
Efficiency	Time complexity tells us how fast an algorithm runs as input size grows. It helps choose the most efficient solution.
Scalability	It predicts if the algorithm can handle very large inputs without becoming too slow.
Comparison	It lets us compare multiple algorithms without actually running them — just by looking at their time complexities.
Optimization	Helps identify bottlenecks and focus optimization efforts on parts of the code that have higher complexity.
Cost Saving	Faster algorithms use fewer resources (CPU time, electricity), saving operational costs, especially in large systems.
User Experience	In real-world applications (like search engines, online apps), faster algorithms lead to better user satisfaction.

BINARY SEARCH

Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log N)$.

To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure should take constant time.

Binary Search Algorithm

Below is the step-by-step algorithm for Binary Search:

1. Divide the search space into two halves by finding the middle index “mid”.
2. Compare the middle element of the search space with the key.
3. If the key is found at middle element, the process is terminated.
4. If the key is not found at middle element, choose which half will be used as the next search space.
5. If the key is smaller than the middle element, then the left side is used for next search.
6. If the key is larger than the middle element, then the right side is used for next search.
7. This process is continued until the key is found or the total search space is exhausted.

Working Of Binary Search

Initially | Find Key = 23 using Binary Search

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Key=23

Low= 0

High = len(arr)-1=10-1=9

High=9

Mid=(low+high)//2 =(0+9)//2. mid=4

arr[mid]=arr[4]=16

Step 1 | Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Key: 23

Low = 0 Mid = 4 High = 9

Since key(23) is greater than the middle element(16),

Low becomes mid+1

Low=mid+1=4+1=5

High remains same

High=9

Mid=low+high//2=(5+9)//2. mid=7

arr[mid]=arr[7]=56

Step 2 | Key is less than the current mid 56.
The search space moves to the left.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Key: 23

Low = 5 Mid = 7 High = 9

Since key(23) is less than the middle element(56),

Low remains same

Low=5

High changes

High=mid -1=7-1=6, high=6

Mid=low+high//2=(5+6)//2. mid=5

arr[mid]=arr[5]=23

Step 3 | If the key matches the value of the mid element,
the element is found and stop search.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Key: 23

Low = 5 High = 6
Mid = 5

Here arr[mid]==key. Therefore index is returned that is key=23 present at index 5.

Time Complexity Analysis Of Binary Search

①

TIME COMPLEXITY ANALYSIS OF BINARY SEARCH ALGORITHM

→ Suppose the list/array has n elements.
→ At each step, the size of the search space is cut in half.

How Many Times Can We Half n ?

After 1st step we divide the search space into two halves that is,

step 1 — $n/2$ elements left
step 2 — $n/2^2$ elements left
step 3 — $n/2^3$ elements left
⋮
step k — $n/2^k$ elements left

when Do we stop?

→ we stop, when the search space size becomes 1 or 0 (zero) (if not found)

so → $n/2^k = 1$

Now

$$\frac{n}{2^k} = 1$$

Multiply both sides by 2^k

$$n = 2^k$$

Apply $\log_2$ on both sides

$$\log_2 n = \log_2 (2^k)$$

$$\log_2 n = k \log_2 2 \quad \because \log_2 2 = 1$$

$$k = \log_2 n$$

Conclusion:

→ the Maximum number of steps required = $\log_2 n$.

→ Hence, Binary search has time complexity $\rightarrow O(\log n)$

BEST CASE $\rightarrow O(1)$ (if the middle element is the key)

AVERAGE CASE $\rightarrow O(\log n)$

WORST CASE $\rightarrow O(\log n)$

Space complexity:

Binary Search Algorithm is an **in-place** algorithm.

This means it doesn't require extra space proportional to the size of the input array.

It only needs a constant amount of additional space.

- **Space complexity = $O(1)$ (constant space).**

When To Use Binary Search

- **The list is already sorted**
Binary search only works correctly on sorted data.
- **You want faster searching**
It's much faster than linear search for large datasets — takes **logarithmic time** ($O(\log n)$).
- **You don't need to modify the list frequently**
If the list keeps changing (adding/deleting elements), you'd have to sort it again before using binary search, which can be expensive.

How to Implement Binary Search Algorithm?

The Binary Search Algorithm can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm

#Program/function to implement Binary Search using iteration

```
def bs(arr,key):  
    low = 0  
    high = len(arr) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
  
        if arr[mid] == key:  
            return mid # Target found  
        elif arr[mid] < key:  
            low = mid + 1 # Search right half  
        else:  
            high = mid - 1 # Search left half  
  
    return -1 # Target not found  
arr = [1,3,5,7,9,11,13]  
key = 13  
result = bs(arr, key)  
if result != -1:  
    print(f"the element {key} found at index {result}")  
else:  
    print(f" the element {key} not found")
```

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

the element 13 found at index 6

Recursive Binary Search Algorithm

```
#Program to implement Binary Search using recursion.
def bs(arr, key, low, high):
    if low > high:
        return -1 # Base case: target not found

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid # Target found
    elif arr[mid] < key:
        return bs(arr, key, mid + 1, high) # Search right half
    else:
        return bs(arr, key, low, mid - 1) # Search left half

arr = [1,3,5,7,9,11,13]
key = 2
result = bs(arr, key, 0, len(arr)-1)
if result != -1:
    print(f'the element {key} found at index {result}')
else:
    print(f'the element {key} not found')

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
the element 13 found at index 6
```

Difference between linear search and binary search.

Feature	Linear Search	Binary Search
Working	Checks each element one by one	Repeatedly divides the list into halves
Requirement	List can be unsorted or sorted	List must be sorted beforehand
Time Complexity	$O(n)$	$O(\log n)$
Best Case	First element is the target ($O(1)$)	Middle element is the target ($O(1)$)
Efficiency	Less efficient for large lists	Very efficient for large, sorted lists
Approach	Sequential	Divide and Conquer
Examples	Searching names in an attendance list manually	Looking up a word in a dictionary book

SORTING

Sorting is the process of arranging data in a specific order, usually ascending or descending, based on a particular criterion. It is a fundamental operation in computer science and is commonly used to organize data for better accessibility, visualization, or further analysis.

Examples of Sorting:

- Alphabetical order: Sorting names in a phone book.
- Numerical order: Arranging numbers in a list from smallest to largest.

TYPES OF SORTING ALGORITHMS

- Bubble Sort
- Selection Sort
- Merge Sort,
- Quick Sort etc...

BUBBLE SORT ALGORITHM

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly traverses the list, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until no swaps are needed, indicating that the list is sorted. The algorithm gets its name because smaller elements "bubble" to the top (start) of the list, while larger elements sink to the bottom (end) with each pass.

ALGORITHM STEPS

Step 1 – Check if the first element in the input array is greater than the next element in the array.

Step 2 – If it is greater, swap the two elements; otherwise move the pointer forward in the array.

Step 3 – Repeat Step 2 until we reach the end of the array.

Step 4 – Check if the elements are sorted; if not, repeat the same process (Step 1 to Step 3) from the last element of the array to the first.

Step 5 – The final output achieved is the sorted array.

Working Of Bubble Sort

EXAMPLE

Now, let's see the working of Bubble sort Algorithm.

To understand the working of bubble sort algorithm, let's take an unsorted array. We are taking a short and accurate array,

Let the elements of array are -

13	32	26	35	10
----	----	----	----	----

First Pass

Sorting will start from the initial two elements. Let's compare them to check which is greater.

13	32	26	35	10
----	----	----	----	----

Here, 32 is greater than 13 ($32 > 13$), so it is already sorted. Now, compare 32 with 26.

13	32	26	35	10
----	----	----	----	----

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like -

13	26	32	35	10
----	----	----	----	----

Now, compare 32 and 35.

13	26	32	35	10
----	----	----	----	----

Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.

13	26	32	35	10
----	----	----	----	----

Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be -

13	26	32	10	35
----	----	----	----	----

Now, move to the second iteration.

Second Pass

The same process will be followed for second iteration.

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be -

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Now, move to the third iteration.

Third Pass

The same process will be followed for third iteration.

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Here, 10 is smaller than 26. So, swapping is required. After swapping, the array will be -

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

Now, move to the fourth iteration.

Fourth pass

Similarly, after the fourth iteration, the array will be -

10	13	26	32	35
----	----	----	----	----

Hence, there is no swapping required, so the array is completely sorted.

Time Complexity Analysis of Bubble Sort

③

TIME COMPLEXITY ANALYSIS OF BUBBLE SORT ALGORITHM

→ The time complexity depends on how many comparisons and swaps are made during the execution of the algorithm

→ Let's take an example to understand the time complexity

EXAMPLE :

list = [13, 32, 26, 35, 10]

PASS - 0

PASS: When all the elements have been compared once, it is called as pass.

13	13	13	13	13
32	32	26	26	26
26	26	32	32	32
35	35	35	35	10
10	10	10	10	35

→ sorted

Maximum possible swaps are : 4

Maximum possible comparisons are : 4

At the end of pass - 0 one element is sorted i.e., 35

PASS - 1

13	13	13	13
26	26	26	26
32	32	32	10
10	10	10	32
35	35	35	35

two elements are sorted
as 35 is already sorted, no need to compare it with 32 ∴ reducing the no. of comparisons.

Maximum possible swaps are : 3

Maximum possible comparisons are : 3

PASS - 2

13 } 13 13

26 } 26 10

10 10 } 26

32 32 } 32

35 35 } 35

sorted, since the next elements are already sorted no need for comparisons. with each pass we reduce the no. of comparisons

Maximum possible swaps are : 2

Maximum possible ^{comparisons} ~~swaps~~ all : 2

PASS - 3

13 } 10

10 } 13

26 26

32 32

35 35

The entire list is sorted.

Maximum possible swaps are : 1

Maximum possible comparisons are : 1

Number of passes required are : $4 = n - 1$

Number of comparisons required : $4 + 3 + 2 + \dots + 1$
 $\downarrow$ in pass-0 $\downarrow$ in pass-1 $\downarrow$ in pass-2 $\downarrow$ until we are left with only 1 comparisons

$$\therefore (n-1) + (n-2) + (n-3) + \dots + 1$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2} \text{ the degree of polynomial is } n^2$$

$\therefore$ the time complexity is $O(n^2)$ in worst case.

⑤

Total number of swaps : $4 + 3 + 2 + \dots + 1$
 $:(n-1) + (n-2) + (n-3) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $= \frac{n^2 - n}{2}$
 $\therefore O(n^2)$ swaps.

BEST CASE : $O(n)$

- When the list is already sorted and no swaps required
- In this case, Bubble sort performs one pass and detects that no swaps were made, terminating early
- The outer loop will still run for $n-1$ iterations, but after the first pass, the swapped flag will prevent further iterations
- $\therefore$ Best case time complexity : $O(n)$, because only $(n-1)$ comparisons are made.

WORST CASE : $O(n^2)$

AVERAGE CASE : $O(n^2)$

- On average, Bubble sort still performs a similar number of comparisons as in the worst case.

LARGEST ELEMENTS

In First pass we get — 1st largest element
" Second " " " — 2nd largest element
" Third " " " — 3rd largest element
:
In k^{th} pass we get — k^{th} largest element.

To get the 2nd largest element we need to perform only 2 passes.

```
#write Program / function to implement Bubble Sort
def bubble(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Flag to detect any swap
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap if elements are in the wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break

arr = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", arr)

bubble(arr)
print("Sorted Array:", arr)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
```

```
#write Program to implement Bubble Sort(without function)

arr = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", arr)
n = len(arr)
for i in range(n-1):
    swapped = False
    for j in range(0, n - i - 1): #
        if arr[j] > arr[j + 1]:
            # Swap if elements are in the wrong order
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True
    # If no two elements were swapped, the list is already sorted
    if not swapped:
        break
print("Sorted Array:", arr)

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
```


Define a python function to find the second largest element in a given list.

A.

- Take a list
- Remove duplicates from the list by converting the list into set using set() method
- Convert the set into list again
- Use .sort() method to sort in ascending order
- Now second largest element will be present at the second index[-2] from the end .

Define a python function to find the second largest element in a given list.

```
def second(nums):  
    unique_nums = list(set(nums)) # Remove duplicates  
  
    if len(unique_nums) < 2:  
        return None # list must contain more than one element  
  
    unique_nums.sort() # Sort in ascending order  
    return unique_nums[-2] # Second last element  
  
# Example usage  
my_list = [10, 20, 5, 8, 20]  
result = second(my_list)  
print("Second largest element is:", result)  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
Second largest element is: 10
```